

ALB analysis

0.1 The ALB data

Getting the data from `gamlss.data`;

```
library(gamlss)
data(ALB)
dim(ALB)
```

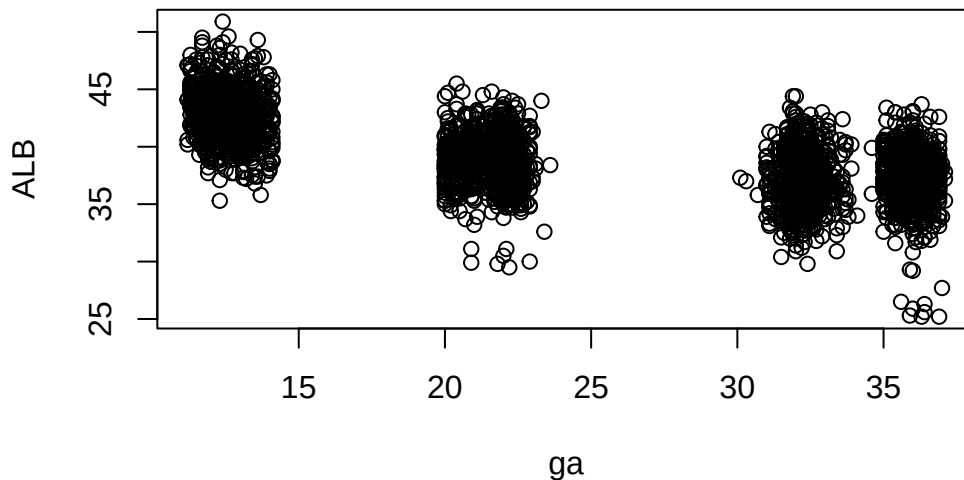
```
[1] 3429    2
```

```
head(ALB)
```

```
      ALB    ga
1 42.1 12.1
2 42.4 11.6
3 43.8 12.7
4 40.3 12.5
5 44.0 12.0
6 44.4 12.9
```

Plotting the data;

```
plot(ALB~ga, data=ALB)
```



The data were collected when woman had their scans at Kings College hospital. That explains why they are caps in gestation age because there are only four different scan on healthy woman during pregnancy. Notice that they here are NOT repeated observations here all 3429 women are different.

💡 The purpose

The purpose of the study is to try to identify patients with low ALB

0.2 Fit a model

Any GAMLSS regression model needs two sets of information to be fitted:

- i) the assumed **distribution** for the response i.e **family=GA**,
- ii) the **selection** of covariates; In fact since we are modelling all distribution parameters as functions of the covariates, we are looking *which covariate affect which distribution parameter*.

Often to start a selection procedure we need a starting model.

We fit two model

- i) a normal model e.g. **family=NO** which works as a staring point for the function `chooseDist()`

```
m00 <- gamlss( ALB ~ pb(ga),
  sigma.formula = ~ pb(ga),
  nu.formula = ~ pb(ga),
  tau.formula = ~ pb(ga),
  family = NO, data = ALB)
```

```
GAMLSS-RS iteration 1: Global Deviance = 15306.02
GAMLSS-RS iteration 2: Global Deviance = 15306.09
GAMLSS-RS iteration 3: Global Deviance = 15306.09
```

ii) a BCTo model e.g. family=BCTo which will work as a starting model for the functionchooseDistCV() (see note below)

```
m0 <- gamlss( ALB ~ pb(ga),
  sigma.formula = ~ pb(ga),
  nu.formula = ~ pb(ga),
  tau.formula = ~ pb(ga),
  family = BCTo, data = ALB)
```

```
GAMLSS-RS iteration 1: Global Deviance = 15198.53
GAMLSS-RS iteration 2: Global Deviance = 15168.6
GAMLSS-RS iteration 3: Global Deviance = 15167.18
GAMLSS-RS iteration 4: Global Deviance = 15164.12
GAMLSS-RS iteration 5: Global Deviance = 15160.44
GAMLSS-RS iteration 6: Global Deviance = 15157.94
GAMLSS-RS iteration 7: Global Deviance = 15156.61
GAMLSS-RS iteration 8: Global Deviance = 15155.96
GAMLSS-RS iteration 9: Global Deviance = 15155.65
GAMLSS-RS iteration 10: Global Deviance = 15155.52
GAMLSS-RS iteration 11: Global Deviance = 15155.45
GAMLSS-RS iteration 12: Global Deviance = 15155.42
GAMLSS-RS iteration 13: Global Deviance = 15155.4
GAMLSS-RS iteration 14: Global Deviance = 15155.39
GAMLSS-RS iteration 15: Global Deviance = 15155.38
GAMLSS-RS iteration 16: Global Deviance = 15155.38
GAMLSS-RS iteration 17: Global Deviance = 15155.38
GAMLSS-RS iteration 18: Global Deviance = 15155.38
```

Note

The `nu.formula = ~ pb(ga)` and `tau.formula = ~ pb(ga)` will be not be used by `chooseDist()` unless more that two parameter are involved in the distribution.

reminder

I have to find out why the function `chooseDistCV()` do not accepts the normal error model like `chooseDist()`.

0.3 Finding a Distributions (training data)

The typical function for finding a distribution is `chooseDist()`. The problem is that the function only finds suitable distributions in the training data using GAIC as the training measure of goodness of fit and therefor it do not tes for over-fitting

```
D1=chooseDist(m00, data=ALB, type="realplus", parallel="snow", ncpus = 10)
```

```
minimum GAIC(k= 2 ) family: BCT
minimum GAIC(k= 3.84 ) family: BCT
minimum GAIC(k= 8.14 ) family: BCPEo
```

For penalties $k = 2$ and $k = 2.84$ the BCT is coming best while for BIC the BCPEo
Here we print a table of the GAIC for all fitted distributions;

```
library(knitr)
kable(D1)
```

	2	3.84	8.14
EXP	31965.45	31975.83	32000.08
GA	15382.23	15396.27	15429.08
IG	15426.08	15440.15	15473.04
LOGNO	15422.55	15436.45	15468.92
LOGNO2	15422.55	15436.45	15468.92
WEI	15592.10	15604.97	15635.04
WEI2	NA	NA	NA
WEI3	15592.11	15604.99	15635.08
IGAMMA	15477.06	15521.24	15624.50
PARETO2	32290.85	32305.98	32341.35

	2	3.84	8.14
PARETO2o	357305.49	357386.43	357575.59
GP	32290.85	32305.98	32341.35
BCCG	15232.64	15255.62	15309.35
BCCGo	15232.94	15255.75	15309.06
exGAUS	15321.58	15339.38	15380.99
GG	15274.98	15292.94	15334.94
GIG	NA	NA	NA
LNO	15422.55	15436.45	15468.92
BCTo	15194.67	15230.82	15315.30
BCT	15194.48	15230.77	15315.58
BCPEo	15203.96	15234.26	15305.06
BCPE	15203.68	15234.19	15305.47
GB2	15240.12	15265.68	15325.42

The problem with the above analysis is that the measure of goodness of fit, **GAIC**, was validated on the training data only. This shows that the model is probably adequate but it does not check for overfitting.

0.4 Finding a Distributions (cross validation)

Next we use a 10th fold cross validation for choosing the distribution. Cross validation hopefully prevents overfitting.

There are two criteria we used here;

- the **test deviance** which is a **global** criterion, that is, it evaluates how the distribution fits overall.
- a **chi-square** criterion which **focuses** on the lower centiles of the distribution.

The new function `chooseDistCV()` is used which is not in the `gamlss` package (yet) so we have to source it;

```
source("chooseDistCVParallel.R")
```

0.4.1 test deviance

First we use a global measurement of goodness of fit criterion the **test deviance** which in the `chooseDistCV()` fit is evaluated using the function. The function `measure_of_goodness_of_fit` needs to be defined before.

```
measure_of_goodness_of_fit <- function(object)
{
  object$CV
}
```

Here to avoid fitting all possible positive range distributions using CV we speed things up by using only a selected set. The set was selected by using the above GAIC table.

```
PP = chooseDistCV(m0, data=ALB, type="extra",
  extra=c("GA", "LOGNO", "BCCGo", "BCPEo", "BCTo"))
```

```
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
```

Warning in predict.gamlss(object, newdata = newdata, data = DatA, what = "nu", : There is a
fit

used to achieve 'safe' predictions

fold 10
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1
fold 2
fold 3
fold 4
fold 5

Warning in additive.fit(x = X, y = wv, w = wt * w, s = s, who = who,
smooth.frame, : additive.fit convergence not obtained in 30 iterations

Warning in additive.fit(x = X, y = wv, w = wt * w, s = s, who = who,
smooth.frame, : additive.fit convergence not obtained in 30 iterations

Warning in RS(): Algorithm RS has not yet converged

fold 6

Warning in RS(): Algorithm RS has not yet converged

fold 7
fold 8
fold 9
fold 10

Warning in additive.fit(x = X, y = wv, w = wt * w, s = s, who = who,
smooth.frame, : additive.fit convergence not obtained in 30 iterations

Warning in additive.fit(x = X, y = wv, w = wt * w, s = s, who = who,
smooth.frame, : additive.fit convergence not obtained in 30 iterations

we can print the order of the fit using;

PP

BCTo	BCPEo	BCCGo	GA	LOGNO
15206.66	15210.04	15244.03	15387.34	15427.75

One could avoid having all the fold output by using parallel computing in R.

```
set.seed(123)
PP1 = chooseDistCV(m0, data=ALB, type="extra",
  extra=c("GA", "LOGNO", "BCCGo", "BCPEo", "BCTo"),
  parallel="snow", ncpus = 10)
```

PP1

BCTo	BCPEo	BCCGo	GA	LOGNO
15212.52	15215.71	15250.28	15390.83	15432.00

There are few things to note here;

- Any cross validation will have results depending on the original computer seed
- Using parallel computation the original seed does not apply in all CPUs therefore the results are different

Fortunately the main results that the BCTo distribution gives the best fit do not change here.

💡 reminder

I use the code below to find out whether using the same random sample makes any difference. It looks that it does not.

```
# set.seed(123)
# sam <- sample(10, dim(ALB)[1], replace=TRUE)
# PP = chooseDistCV(m0, data=ALB, type="extra",
#   extra=c("GA", "LOGNO", "BCCGo", "BCPEo", "BCTo"), rand=sam)
# PP
```



```
# set.seed(123)
# PP1 = chooseDistCV(m0, data=ALB, type="extra",
#                   extra=c("GA", "LOGNO", "BCCGo", "BCPEo", "BCTo"),
#                   parallel="snow", ncpus = 10, rand=sam)
```

```
# PP
# PP1
```

0.4.2 chi-squared (focus)

First bring the functions in;

```
source("Chi2.R")
```

Define the focus measure of goodness of fit;

```
measure_of_goodness_of_fit <- function(object)
{
  cal_chi2(object$residCV, xvar=ALB$ga, breaks = c(0,17,25,34.5, 50), tail.percent = 5, si
}
```

Now choose distribution using CV with no parallel computaions.

```
PPP = chooseDistCV(m0, data=ALB, type="extra",
                  extra=c("GA", "LOGNO", "BCCGo", "BCPEo", "BCTo"))
```

```
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1
fold 2
fold 3
fold 4
```

fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1
fold 2
fold 3
fold 4
fold 5
fold 6
fold 7
fold 8
fold 9
fold 10
fold 1

Warning in RS(): Algorithm RS has not yet converged

fold 2
fold 3
fold 4

Warning in RS(): Algorithm RS has not yet converged

fold 5
fold 6
fold 7
fold 8
fold 9
fold 10

Now choose distribution using CV with parallel computaions.

```
PPP1 = chooseDistCV(m0, data=ALB, type="extra",  
  extra=c("GA", "LOGNO", "BCCGo", "BCPEo", "BCTo"),  
  parallel="snow", ncpus = 10)
```

Now print the results

PPP

BCTo	LOGNO	GA	BCCGo	BCPEo
3.020492	4.513079	4.759659	12.239766	13.393470

PPP1

LOGNO	GA	BCTo	BCCGo	BCPEo
3.400714	4.074632	5.070350	13.194654	13.194654

 reminder

This interesting the same selection procedure points to a different distribution